

1주차 - Preliminaries

p.1

#프로그래밍 언어란?

언어 : 영어, 한국어, 일본어...

프로그래밍 언어 : FORTRAN, C, COBOL

기계어 > 어셈블리어 > 고급언어 > Very HLL

#프로그래밍 언어를 왜 배우는가?

Imperative(Procedural) 언어 : 폰 노이만 컴퓨터

Declarative(Applicative) 언어 : non-폰 노이만. Prolog, LISP 등...

#이 수업에서는 무엇을 배우는가?

Data : int, char, float, array, structure...

Control : if_then_else, for_loop...

Subprogram : procedure vs function

p.2-p.3 그림 참고

p4.

#프로그래밍 언어들의 개념을 배우는 이유

1. 아이디어를 표현할 수 있는 능력의 증가.
2. 적절한 언어를 선택할 수 있음.
3. 새로운 언어를 배울 수 있는 능력의 향상
4. 구현의 중요성에 대한 더 나은 이해
5. 이미 알고 있는 언어에 대한 더 나은 이해
6. 컴퓨팅의 전반적인 발전

#프로그래밍의 영역

* 과학 용도 : 많은 부동소수점 연산이 필요. 1950-60년대 흔했음. Fortran.

* 비즈니스 용도 : 정교한 입출력, 10진수 계산 필요. 비즈니스용 - COBOL

* AI : 기호 처리 특성화. LISP, PROLOG

* 시스템 프로그래밍 : 효율성 중시. 주변장치 드라이버 SW를 작성할 정도의 low-level 기능이 필요함. C언어

* 웹 프로그래밍 : HTML + 스크립팅 언어 -> 동적인 콘텐츠 Java

#4가지 언어의 평가 기준

- | | |
|----------------|-----|
| 1. Readability | 가독성 |
| 2. Writability | 작성력 |
| 3. Reliability | 신뢰성 |
| 4. Cost | 비용 |

1. Readability 가독성

: 프로그램의 가독성이 높으면 유지관리가 쉬워지기 때문. SW 생명주기 개념이 등장한 후 도입.

*Overall Simplicity 전반적인 단순성

기초 요소(components)들이 적은 언어가 더 배우기 쉽다.

Feature multiplicity : 한 기능을 하기 위한 방법이 여러 개 ex) `count++` `count=count+1`

->적을수록 좋음

Operator overloading : 한 연산자가 여러개의 의미를 가짐 ex) `+` 기호(산술합, 문자열 합치기)

반대로, 너무 단순해도 가독성이 안 좋다. ex) 어셈블리어

*Orthogonality 직교성

언어를 구축할 때 결합할 수 있는 경우가 얼마나 많은가?

예외가 적을수록 직교성이 높다.

언어의 다양한 기능이 조합될 때도 예상 가능한 방식으로 작동하는가?

ex) 직교성이 나쁜 예시 : C언어에서 포인터 변수와 그냥 변수를 더했을 때의 결과

*Control Statements 제어문

위에서 아래로 읽히는 코드가 이해하기 쉽다.

인접하지 않은 곳으로 점프하는 것은 이해하기 어렵다.

ex) GOTO문은 가독성을 나쁘게 한다.

*Data Types and Structures 자료형과 데이터 주고

언어에서 적절하게 정의된 자료형과 데이터 구조를 제공해야 가독성이 좋다.

ex) `record`, `bool`, `enum`

*Syntax Considerations 구문 설계

Identifier forms

Special words 예약어

Form and meaning `GO TO (10,20,30), I`

`GO TO I ,(10,20,30)` => 이 두 개가 의미가 완전 다르다. 가독성이 나쁘다.

한 가지 명령은 한 가지 의미만 있어야 가독성이 좋다!

2. Writability 작성력

: 언어로 얼마나 쉽게 프로그램을 만들 수 있는지

* Simplicity and Orthogonality

단순하고 직교성이 높으면 언어를 작성하기 쉽다.

* Support for Abstraction

추상화를 지원. 디테일한 부분을 무시하더라도(몰라도) 복잡한 구조나 작업을 할 수 있는 것.

<-> Process abstraction 프로세스 추상화 : subprogram(함수같은 것)을 사용

<-> Data abstraction 데이터 추상화 : Record type. C언어에서의 구조체

* Expressivity 표현성

같은 동작을 하지만, 상대적으로 편리한 기능.

ex) C에서 `count++`은 `count=count+1`보다 더 편리함.

3. Reliability 신뢰성

: 모든 조건에서 잘 수행하는지.

* Type Checking 타입 검사

type compatibility(호환성)를 검사하는 것

런타임에 검사하는 것은 expensive함. 컴파일 시간에 하는 것이 바람직.

-> C언어는 타입 검사가 없음. 신뢰성이 좋지 않음

-> Pascal은 대부분 컴파일 시간에 타입 검사. 일부는 런타임에 함.

-> Java는 타입 검사를 아주 엄격하게 함.

*Exception Handling 예외 처리

런타임 오류 및 비정상적인 조건 등을 고치는 조치. 신뢰성을 높임

-> C언어에서는 예외처리가 필수가 아님. 안 해도 됨

-> Java는 예외처리에 엄격. 하지 않으면 컴파일이 안 됨.

*Aliasing 별칭

동일한 메모리 공간을 서로 다른 두 가지 방법으로 접근하는 것

신뢰성에 좋지 않음. 위험할 수 있음.

-> C언어의 포인터

* Readability and Writability

두 가지가 낮으면 유지보수하기 힘들고, 부자연스럽게 구현하게 될 수 있음.

4. Cost 비용

* 프로그래머가 언어를 쓸 수 있게 교육하는 비용

* 코드를 작성하는 데 드는 비용

* 컴파일하는데 드는 비용

* 실행하는 데 드는 비용

* 유지보수(maintaining)하는 데 드는 비용

5. Portability

얼마나 많은 플랫폼에서 그 언어의 컴파일러가 작동할 수 있는가?

6. Generality 범용성

얼마나 다양한 분야에서 언어를 쓸 수 있는가?

7. well-definedness

언어에 대한 문서화가 얼마나 잘 되어 있는가?

#컴퓨터 아키텍처

* Imperative languages 명령형 언어 - 폰 노이만 컴퓨터

문제를 어떻게 풀 것인가에 초점이 맞추어진 언어. 하드웨어 구조와 작업에 대한 높은 수준의 추상화

-> 변수, 배정문, 반복문, 표현 등의 특징이 있음.

문제점 : 병목현상 CPU-메모리 간 속도차이 => Cache를 도입. but 완전히 해결할 수 는 없음

Cache hit ratio

* Declarative languages 선언형 언어 - non-폰 노이만 컴퓨터

배정문, 반복문 없음. 함수형 언어의 효율적인 실행이 가능할 듯.

아직까지 개발되지 않음.

Non-폰 노이만 컴퓨터 : 순차 제어, 변수 개념이 없을 수도. 이름과 상수 간의 강한 구속력.

#프로그래밍 방법론

- * Structured Programming 구조화된 프로그래밍 70년대 초
Top-down 디자인. Go-to 최소화.
절차지향 프로그래밍
PASCAL, ALGOL
- * Data-Oriented Programming 70년대 말
데이터 추상화(Abstraction)를 지원함.
Simula67, CLU, Modula-2, C++, Ada
- * Object-Oriented Programming 80년대 초
데이터 추상화 + 캡슐화 + 동적 타입 바인딩
이미 있는 소프트웨어를 재사용
Smalltalk, CLOS, C++
- * Process-Oriented Programming
프로그램의 동시성(Concurrency)를 중시함. 동일한 시간에 여러개의 프로세스를 실행할 수 있게 함.
Concurrent Pascal, Ada, Java의 스레드

#언어 디자인 Trade-offs

- * 신뢰성 <-> 실행 비용 ex) C vs Java 타입 검사를 하면 시간이 들지만 신뢰성은 올라감
- * 간결함 <-> 이해하기 쉬움 ex) 엄청 짧게 쓸 수 있지만 반대로 이해하기 어려울 수 있음.
- * 유연성 <-> 안전성 ex) C언어의 포인터
- * 유연성 <-> 효율성 ex) C언어의 포인터

#컴파일러 vs 퓨어 인터프리터 vs 하이브리드

- * 컴파일러
고급->저급 언어로 번역본을 미리 만들어 두는 것.
실행이 굉장히 빠름. but 수정을 하면 다시 컴파일을 해야 함.
C, COBOL, Ada
- * 퓨어 인터프리터
실행시간에 한줄한줄 읽으며 해석을 함.
컴파일러에 비해 80배 정도 느림.
메모리 등 리소스를 많이 잡아먹음
컴파일러보다 구현하기가 쉬움.
파이썬, JS, APL
- * 하이브리드 시스템
고급 언어 -> 중간언어 -> 저급언어
컴파일러와 인터프리터를 둘 다 사용
Java : Java ->(컴파일러) Java Bytecode ->(인터프리터) 기계어
>> 언어의 범용성을 위해서. 가상머신(JVM) 위에서 돌아감.

#전처리기 : 프로그램을 확장시킴. #include 소스->전처리->컴파일->링크->실행파일

#프로그램 환경

- : 소프트웨어 생산에 이용되는 도구의 모음
ex) 파일 시스템, 편집기, 링커, 컴파일러, 디버거, UI...

2주차 - Evolution of the Major Programming Languages

FORTRAN (Formular translator)

IBM 704을 개발하기 위한 언어(Machine dependent). - 하드웨어적으로 **부동소수점** 계산이 가능했음.

FORTRAN 0 : 실제로 구현 X

FORTRAN I : 1957

- * 컴퓨터 리소스가 작고, 믿을만하지 않았음
- * 과학 용도로 사용
- * 프로그래밍 방법론이나 도구가 없음 -> 천공카드
- * 기계의 효율이 제일 중요한 고려사항이었음.

특징 :

동적 저장 X

좋은 array 핸들링과 counting loop가 필요했음

I, J, K, L, M, N으로 시작하는 변수는 int. 나머지는 float

문자열 없음. 강력한 입출력 없음.

의의 :

★최초로 컴파일러를 사용하는 고급 언어★ or 최초의 고급 언어

이름 최대 6자

Post-test counting loop (DO 문) 먼저 실행하고 나중에 검사

입출력 형식이 정해져 있었음.(formatted)

세 가지 선택 구문

데이터타입을 지정하는 구문이 없음. 선언할 때 이름으로 구분.

부분 컴파일이 안 됨. 무조건 전체.

18명이 만들어 1957년에 컴파일러 배포.

400줄 넘어가면 작동을 안함.

코드가 **매우 빠름** C언어보다 2배 빠름

빠르게 널리 쓰이게 됨. 인기 언어.

Fortran II

부분 컴파일 가능(Independent compilation)

버그 수정

Fortran IV

타입을 명시적으로 선언 가능

논리적 조건문(logical selection) 가능 bool <- 원래는 숫자만 가능

subprogram을 매개변수로 사용 가능

1966년 ANSI standard가 됨.

Fortran 77

문자열(Character string) 지원

논리적 반복문(logical loop) 가능

If-then-else else 가 생김.

Fortran 90

모듈, 동적 배열, 포인터, 재귀, CASE 구문, 매개변수 타입 검사

Fortran 최신 버전

Fortran 95, Fortran 2003

Fortran에 대한 평가

- * 매우 최적화가 잘 된 컴파일러(90이전 버전) b.o. 모든 변수가 런타임 이전에 fix되어 있음
- * *lingua franca* ; 컴퓨터 세계에서의 공용어

#LISP : 함수형 언어 LISP Processing language

★최초의 함수형 프로그래밍 언어★

MIT의 McCarthy에 의해 개발됨.

AI 연산에 어울리는 언어를 만들기 위해서 만들어짐.

구문은 *lambda calculus* 람다 함수 기반으로 만들어짐

평가 :

AI 언어 부분에서 절대적임.

느린 실행 시간. 최초로 인터프리터 언어. 지금은 컴파일러 방식으로 바뀜

Common LISP, Scheme 같은 파생언어(dialects) 등장

예시 :

(+3 5) => 8

(-100 7) => 93

(* 2 3) => 6

Prefix 전위 표기법을 사용했다.

#Scheme

LISP보다 작음

Static scoping. 비지역변수에 접근 가능

함수를 first-class entities. => 함수를 매개변수로 사용 가능

LISP 보다 문법을 간단하게 만들었음. 교육용으로 사용

#Common LISP

LISP의 dialects(파생언어)들을 모두 하나로 통합한 언어

크고 복잡함.

#ALGOL

★universal language(범용적인 언어)를 목표로 만들어진 언어! 기계 종속적이지 않음. ★

Fortran은 IBM 컴퓨터에서만 작동했음.

목표

- * 수학적 표기법과 비슷하게 하자
- * 알고리즘을 표현하기 좋게 하자
- * 각자의 기계에 맞게 해석되도록 하자

특징

데이터 타입을 형식화

이름이 길어도 상관 없음

배열의 차수에 제한이 없음

매개변수가 모드에 따라 구분되었다

배열이 brackets []으로 표현되었다.

복합문(여러 줄)을 지원한다. (begin ... end)

현대는 { } 중괄호

문장 구분을 위해 세미콜론을 쓴다

assign 연산자를 := 로 한다.

현대는 =

if 다음에 else-if 가 올 수 있음

IWO 입출력 기능은 없음! b.o. 하드웨어 종속적이게 하지 않기 위해서.

ALGOL 58

구현은 되지 않음. IBM이 지원하다 지원을 끊어버림.

ALGOL 60

ALGOL 58을 수정했음

추가 :

블록 구조(local scope)

두가지 파라미터 전달 방식

subprogram 재귀

스택 동적 배열

but, 여전히 I/O와 문자열 handling은 없음.

성공한 것 :

알고리즘의 표현의 기준이 되었음.

★이후의 모든 imperative languages(명령형 언어)의 기반이 됨★

최초의 기계 독립적인 언어.

syntax가 공식적으로 정의된 첫 번째 언어 - BNF

실패한 것 :

널리 쓰이지 못함. (특히 미국)

입출력을 구현하기 어려웠음.

너무 유연함(구현하기 어려움)

Formal syntax description(BNF)

IBM의 지원 부족

COBOL

최초로 비즈니스용으로 개발된 언어

기존 FLOW-MATIC을 기반으로 개발됨.

FLOW-MATIC의 기능 :

이름은 하이픈 포함 최대 12자

연산자를 영어 이름으로 만들(not +, -)

데이터와 코드를 완전히 분리함

모든 구문이 동사로 시작함.

1959년 펜타곤에서 미팅.

디자인 목표 : 간단한 영어로. 사용하기 쉬워야 함. 컴파일러의 문제에 의해서 제약(bias..?)가 있으면 안 됨.

컴퓨터 제조업자+ DoD(국방부 관련자)가 모여서 만들.

디자인 문제 : 산술연산 표현, 아래첨자를 어떻게 해야 일반인이 알아보기 편할 것인가?

의의 :

최초로 매크로가 포함된 고급 언어

계층적 자료 구조(Records; 구조체와 비슷)

Nested selection statements 중첩 조건문

긴 이름. 하이픈 포함 30자까지 가능.

데이터와 코드 분리

DoD(국방부)에서 요구한 첫 번째 언어. DoD의 지원으로 성공함.
현재도 쓰이고 있음.

BASIC

Dartmouth 대학 Kemeny & Kurtz 2명이서 만듦

유명한 파생어 : Visual Basic

최초로 시분할(Time Sharing)을 도입한 언어

PL/1 Programming Language 1

IBM, SHARE이 설계

1964년의 상황

-과학 컴퓨팅 : Fortran을 사용하는 SHARE 유저 그룹

-비즈니스 컴퓨팅 : COBOL을 사용하는 GUIDE 유저 그룹

>> 서로의 영역을 침범해야 하는 경우가 생김.

IBM에서 3명, SHARE에서 3명이 만남.

IBM의 Fortran IV의 확장판을 컨셉으로 개발함. NPL로 시작해 => PL/1으로 이름 바꿈

의의 :

동시에(concurrency) 실행 가능

첫 예외 처리

Switch 조건 재귀문

switch문

★첫 포인터 자료형 도입★

배열의 일부분만 참조 가능(array cross sections)

문제 :

새로운 기능들이 잘 디자인되지 못함

너무 많고 복잡함

>>그래도 어느정도의 사용자를 끌어들이고, 절반정도 성공을 거둠

APL & SNOBOL 최초의 동적 언어

동적으로 타입, 동적 저장공간 할당 지원

변수가 타입이 정해져있지 않음. 파이썬처럼 변할 수 있음.

값이 assign되기 전까지 저장공간에 할당되지 않음

APL :

표현력은 좋지만, 코드를 이해하기 굉장히 어려움(가독성 ▼)

아직도 소수는 사용 중

SNOBOL :

벨 연구소 Farber, Griswold, and Polensky

문자열 패턴 매칭(문자열에서 특정 문자열 찾기)에 강력한 성능

#SIMULA 67 최초의 데이터 추상화

노르웨이 Nygaard, Dahl가 시스템 시뮬레이션을 위해 만들

처음 Coroutines(일종의 subprogram) 도입

처음 클래스, objects(객체), 상속 도입

#ALGOL 68 Orthogonality

Orthogonality 개념을 기반으로 디자인

의의 : 사용자 정의 데이터 구조, 동적 배열(flex arrays)

>> 많이 쓰이지는 않았지만, Pascal, C, Ada에 큰 영향

#Pascal 1971

Wirth에 의해 개발(ALGOL 68 개발자)

교육용으로 개발. 70s-90s 교육용으로 많이 쓰임

#C 1972

벨 연구소. 시스템 프로그래밍을 위해 개발됨

BCLP, B, ALGOL 68의 영향을 받음

강력한 연산자, but 타입 검사는 하지 않음.

UNIX 운영체제를 통해서 널리 퍼짐

많은 사용자

#Prolog 1970s

마르세유대 Comerauer, Roussel

형식 논리에 기반. 추론 프로세스

비효율적, 적용 범위가 적었음.

#Ada

디자인하는데 많은 자원과 시간을 투입함. DoD 참여

Ada 이름은 최초의 프로그래머 이름을 따.

의의 :

패키지 - 데이터 추상화 지원

예외 처리 - 세련되게 만들

generic program units

concurrency - through the tasking model

언어 완성 후 컴파일러를 만드는데 5년 걸림

#Ada 95 최초의 객체지향 언어

의의:

객체지향 지원

공유된 데이터에 대한 더 나은 제어 메커니즘 (Private Public 제어범위 설정 가능)

동시 기능

더 많은 라이브러리

잘 만든 언어지만 C++에 밀려서 잘 사용하지 않음.

#Smalltalk

Xerox 직원 Alan Kay

최초의 완전한 객체지향 언어(데이터 추상화, 상속, 동적 바인딩)

GUI 지원

진보된 OOP

#C++

벨 연구소 Stroustrup

C와 SIMULA67을 발전시킴

예외처리 제공

절차지향과 객체지향 모두 지원

파생어 : MS의 Managed C++

운영체제, 게임 등에 사용

#Java

SUN에 의해 개발. C/C++는 임베디드에 적합하지 않았음.

C++ 기반으로 개발.

의의 :

C++에서 struct, union, enum, 포인터 등 빼버림

OOP만 제공!

concurrency를 하기 위해 applets 지원 > 현재는 thread

Portable : Java 가상머신. JIT(Just in Time; 실시간) 컴파일러

#스크립트 언어

웹 개발하는데 사용

특징 : 전부 인터프리터 사용

#Perl

변수형이 어떤 글자로 시작하는지에 따라서 변수형이 암묵적으로 정해짐. Reliability 낮음

UNIX 시스템 관리 언어로도 쓰임

#JavaScript

넷스케이프에 의해 개발

클라이언트 side에서 작동하는 코드

Java와는 무관

#PHP

서버 side에서 작동하는 코드

웹을 통해서 데이터베이스에 액세스하는 언어.

#Python

객체지향 인터프리터 언어

타입 검사를 함. 동적 타입

리스트, 튜플, 해시 지원

시스템 개발 불가능.

#Lua

브라질

확장성을 가지는 객체지향

게임 개발에만 사용됨

table이라는 한 개의 자료형만 있음

#Ruby

일본 유키히로 마츠모토
순수 객체지향 언어. 클래스와 객체 모두 동적
확장 가능.

C#

마이크로소프트 .NET에서 개발
Common Type System : 여러 언어에서 사용하는 공통 클래스 라이브러리

#XSLT

eXtensible Markup Language: XML을 HTML로 번역하기 위해서 만들어진 **마크업** 언어
반복문과 같은 프로그래밍 언어의 특성을 가지고 있음

#JSP - 실제 자바랑 관련이 있는 언어

Java Server Pages : 동적 [Sublet 서블릿 : 서버 side에서 작동하는 Java 언어. HTML 언어에 의해서 요청됨
별다른 수정 없이 Java로 실행 가능

3주차 - Describing Syntax and Semantics

Syntax 구문

: expressions, statements, program units의 구조

Symantics 의미론

: expressions, statements, program units의 의미

>> Syntax + symantics => 언어의 정의

Sentence 문장

: 몇 글자로 이루어진 문장(string of characters)

Language 언어

: 문장들의 집합

Lexeme 어휘

: 언어의 가장 낮은 수준의 구문 단위.(의미를 가지는 최소 단위) ex) 변수이름, =, 2, *, ;

Token 토큰

: 각 Lexeme을 카테고리화 한 것 ex) 식별자, 등호, 정수, 곱연산자, 세미콜론

Context-free grammars == BNF(Backus-Naur Form)

Noam Chomsky가 Context-free grammars를 개발함. 바쿠스-나우어 품도 비슷하게 개발됨.

Context-free grammars와 Backus-Naur Form은 다른 사람에 의해 개발됐지만 같은것이다!

: 문자열 패턴을 생성하는 데 사용되는 재귀적 재작성 규칙의 집합

★좌변에서 우변으로 전개되는 문법 구조

Terminal : Lexemes이나 Tokens

LHS(Left-hand side)은 nonterminal, RHS(Right-hand side)는 terminals이나 nonterminals들로 이루어짐.

Nonterminal은 <>로 둘러싸여 있음.

문법 : 규칙을 가지는 유한개의 집합

Start symbol : 문법의 nonterminal의 특별한 요소

* 구문 목록은 재귀를 사용할 수 있음.

* derivation(유도) : 시작 기호로 시작하여 문장으로 끝나는 규칙의 반복적인 적용(모두 터미널로 끝남)

```
<program> => <stmts> => <stmt>
=> <var> = <expr>
=> a = <expr>
=> a = <term> + <term>
=> a = <var> + <term>
=> a = b + <term>
=> a = b + const
```

```
<program> → <stmts>
<stmts> → <stmt> | <stmt> ; <stmts>
<stmt> → <var> = <expr>
<var> → a | b | c | d
<expr> → <term> + <term> | <term> - <term>
<term> → <var> | const
```

주의할 점.

1. 한 번에 하나씩. 건너뛰면 안 됨.
2. 바꾸는 순서는 한 방향으로. 왼쪽->오른쪽으로.

Sentential form(문장 형태) : 유도 과정에서 각각의 줄

Sentence : **터미널로만** 이루어진 sentential form

Leftmost derivation(최좌단 유도) : 맨 왼쪽에 있는 것부터 유도하는 방식

Parse Tree

: 유도를 계층 구조에 따라서 하는 것

ambiguous (모호함) : 문법이 모호하다는 것은 한 문장이 두 가지 이상의 parse tree로 그려질 수 있을 때.

****이 문장이 모호함을 증명하시오! -> Parse Tree를 그려서 두 개가 나오면 Ambiguous하다!**

모호하지 않은 문법

1. 연산자 간 우선순위를 지정하면 모호성을 없앨 수 있다.
2. 결합법칙을 통해서 연산하는 순서를 고정시키면 모호성을 없앨 수 있다.

#Extended BNF

- * 선택적인 부분을 대괄호로 표현함. 있어도 되고 없어도 되는 부분. ~~ ident [(<expr_list>)]
- * 대체적인(둘 중 선택)하는 부분은 괄호 안에 |로 구분. <term> -> <term> (+|-) const
- * 반복문을 중괄호로 표현함. <ident> -> letter {letter|digit}

• BNF

```
<expr> → <expr> + <term>
        | <expr> - <term>
        | <term>
<term> → <term> * <factor>
        | <term> / <factor>
        | <factor>
```

• EBNF

```
<expr> → <term> {(+ | -) <term>}
<term> → <factor> {(* | /) <factor>}
```

#Semantics 의미론

:작성된 문장의 의미를 결정하는 것. 널리 받아들여지는 하나의 표기법이나 공식적인 형태는 없음.

의미론이 필요한 이유

프로그래머들은 문장이 무슨 의미하는지를 알아야 한다.

컴파일러 작성자는 각 구문의 동작을 정확히 이해해야 한다

정확성 증명이 가능해야 함. - 프로그램이 의도한대로 동작하는지 수학적으로 증명 가능

컴파일러 생성이 가능해야 함. - 명확한 의미론을 기반으로 컴파일러를 자동으로 생성이 가능해짐

모호함과 불일치(일관되지 않음)을 감지할 수 있음.

#Semantics의 세 가지 접근

1. Operational Semantics(연산 의미론) : 실행과정이 직접 설명됨.

abstract machine(가상의 기계)를 정의하고 기계의 상태에 유도하는 전환을 설명함으로써 의미 부여.

2. Axiomatic Semantics(공리 의미론) : 논리적 공리를 설명함으로써 구문에 의미를 부여

수학적 논리 프로그램을 실행하면서 어떤 변화가 생기는지, 변수-상수들 간의 의미가 어떻게 되는가 관계를 분석하며 의미를 부여

프로그램 검증, 증명에 사용.

Assersion(단언) : 논리 프로세스 - 실행시점의 참인 변수간의 관계와 제약을 명시하는 역할을 함.

Postcondition(사후조건) : 문장 뒤의 조건

Weakest precondition(가장 약한 사전조건) : 사후조건을 보장하는 가장 제약이 약한 사전조건.

프로그램 검증(증명)과정 :프로그램의 마지막 문장의 사후 조건을 프로그램 전체의 사후조건이라 하고, 그 조건의 마지막 문장의 가장 작은 사전 조건을 계산한 후 이전문장의 사후조건을 마지막 문장의 가장 작은 사전 조건으로 한다. 이런식으로 거슬로 올라가 첫 번째 문장의 가장 작은 사전 조건이 프로그램의 명세와 동일하면, 프로그램이 올바르다.

추론규칙(Inference rule) : 한 문장의 참을 다른 assertion의 값에 기반하여 추론하는 것

S1, S2가 참이면, S1과 S2가 연결되어도 참이다....?

• Pre-, post form: {P} statement {Q}

{P1} S1 {P2}

{P2} S2 {P3}

• An example

- a = b + 1 {a > 1}

- One possible precondition: {b > 10}

- Weakest precondition: {b > 0}

$\frac{\{P1\} S1 \{P2\}, \{P2\} S2 \{P3\}}{\{P1\} S1; S2 \{P3\}}$

3. Denotational Semantics(표기 의미론) : 수학적 함수 형태로 사용하여 설명. 메타언어로 공식화 될 수 있음.

Attribute grammar(속성 문법) : 언어의 구문과 의미론을 모두 설명할 수 있는 형식주의

연산, 공리 방법은 의미론을 설명이다...

5장 - Names, Bindings, Type Checking, and Scopes

Imperative Languages 명령형 언어

폰 노이만 컴퓨터 구조의 추상화한 것이 명령형 언어이다.

<=> memory : instruction과 data를 저장

<=> CPU :

변수 : 메모리 셀의 추상화. 속성(개체)들의 집합.

자료형, scope, life time, 타입 검사, 초기화.....

자료형이 얼마나 현실 세계에 잘 맞는가?

Names 이름

: 개체(변수, labels, subprograms...)를 구분하는 문자열(string of characters)

디자인 고려사항

- * 이름의 최대 길이는 얼마로 해야 하는가?
- * connector/underscore (-/_)가 이름에 포함될 수 있는가?
- * case sensitive 대소문자 구별을 하는가?
- * special words, reserved words, keywords인가?

이름 길이

초창기 언어 : 1글자

FORTRAN 1 : 최대 6글자

COBOL : 최대 30글자

C, C++ : 제한 없음

Connector/Underscore

1970-80년대 my_stack is_full

현대 myStack isFull <- Camel Notation

Case Sensitive

C/C은 대소문자를 구분한다.

=> Readability의 관점에서 좋지 않음!

Special words - Key/Reserved Words

Special Words : 특정한 작동을 하도록 만들어진 단어. 구문과 프로그램의 구문 부분을 구분하는데 사용됨

Keywords words : 특정 맥락에서 특별한 의미를 갖는 단어

FORTRAN에서 Real apple은 apple이라는 '실수' 변수를 선언하는 것이고

real=3.4 는 변수 이름으로 real을 쓰는 것임

Reserved Words : 이름으로는 쓸 수 없는 특별한 단어(Special words)

=> 언어 디자인을 할 때 키워드보다 예약어가 낫다!

#Variables 변수

: 메모리 셀의 추상화

주소 : 연관되어 있는 메모리의 주소

지역 변수 : 같은 이름이어도 다른 공간에 있으면 다른 주소로 연관되기 때문에 같은 이름을 쓸 수 있다.

aliases 별칭 : 1개 이상의 변수 이름이 한 메모리 위치로 접근하는 것. => Readability 측면에서 나쁨

Type 타입

값의 범위, 연산자의 집합(어떤 연산이 가능한지)를 정하는 것.

ex) FORTRAN의 Integer는 -32768~-32767이 범위이고 산술 연산이 가능

Value 값

추상화된 메모리 안에 있는 내용물(contents)

l-value : 변수의 주소

r-value : 변수의 값

a=a+1 좌변의 a는 l-value, 우변의 a는 r-value

Binding 개념

바인딩

: 속성과 개체(entity), 연산자와 기호 간의 **연관**; **관련있게 만드는 것**

Binding time : 바인딩이 발생하는 시간

* 언어 디자인 시간

'*' 기호를 산술곱과 연관시킴(binding 함)

* 언어 구현 시간

특정 값의 가능한 값의 범위를 어디까지 할 것인가?

* 컴파일 시간

어떤 변수가 특정 데이터 타입으로 binding

* 링크 시간

라이브러리의 subprogram을 불러와 binding

* 로드 시간

선언한 변수를 실제 메모리에 올리는 것

a variable may be bound to a storage cell **when the program is loaded into memory**

* 실행 시간 runtime

변수에 대한 값 binding. 실행 중에 호출되는 경우.

a variable **in a procedure** may be bound to a storage cell **when the procedure is actually called**

ex) int count ; <- 컴파일 시간에 count가 int 형으로 bind됨

...

count = count*5;

sub1(count);

....

sub1(int aa){....}

Q) Count가 가질 수 있는 값의 범위는 언제 binding 되는가?

A) 언어 구현 시간

Q) 3줄의 * 기호가 산술곱으로 binding된 건 언제인가?

A) 컴파일 시간

>> 실제로 곱셈으로 결정되는 순간은 컴파일 되는 순간이기 때문!

Q) count 변수의 값은 언제 binding 되는가?

A) 실행 시간

Q) sub1(count)는 언제 binding 되는가?

A) 실행 시간

Q) sub1(int aa)는 언제 binding 되는가?

A) 로드 시간

>> ???????

#변수 속성에 대한 바인딩

static 정적 : 실행 시간 전에 일어나고, 실행 동안에는 끝까지 바뀌지 않는 것.

dynamic 동적 : 실행 시간 도중에 일어나거나, 실행 중에 바뀔 수 있는 것.

#Type Binding 타입 바인딩

Static Type Binding(변수 선언) - 실행시간 도중에 자료형이 바뀌지 않음

명시적 **explicit** 선언 : 변수를 특정한 자료형으로 선언하는 것. 60년대 이후로 대부분의 언어에서 해당.

묵시적 **implicit** 선언 : convention에 의해서 자료형이 결정됨. FORTRAN(I,J,K,...), BASIC, PL/1 등

Dynamic Type Binding

: 할당하는 구문에서 변수의 값이 할당될 때 binding이 됨.

주로 인터프리터 언어에서 쓰임. => 파이썬이 대표적.

장점 : 유연함

단점 : 컴파일 시간에 오류를 찾아내기가 힘들, 타입 검사하는데 비용이 들고, 태그에 공간이 쓰이게 됨.

타입 추론 Type Inference

ML 언어에서 `fun circum(r) = 3.14*r*r`에서는 좌변으로 어떤 값이 넘어갈까...? 추론하는 것

Generic Programming(Parametric polymorphism)

: 특정 타입에 의존하지 않고, 하나의 변수가 여러 다른 데이터 타입을 가질 수 있게 하는 프로그래밍 스타일.

코드의 재사용성을 늘릴 수 있음. ML이라는 언어에서 제시됨.

#Storage Binding and Lifetime

저장공간 바인딩과 생명

: 변수의 수명은 변수가 특정 메모리 위치에 바인딩되는 시간

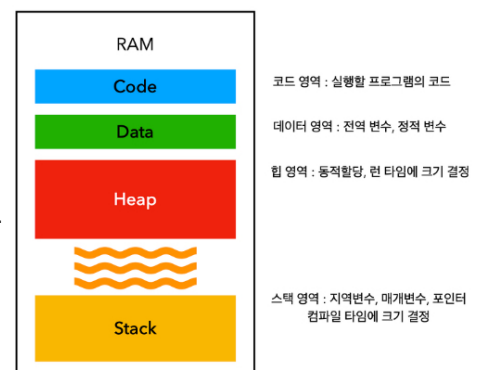
Static Variables (정적변수)

: 프로그램 실행 전에 메모리 셀에 바인딩되고, 끝날 때까지 바인딩됨.

ex) global variables 전역변수, history sensitive variable

장점 : 효율적(변수 할당하는데 런타임에 시간이 들지 않음)

단점 : 유연성 감소(재귀 recursion을 구현할 수 없다)



Semidynamic Variables (Stack Dynamic Variable)

: 선언문(declaration statement)을 통해 storage에 바인딩되지만, 타입은 정적으로 바인딩 됨

ex) local variable **지역변수**

장점 : 같은 메모리 공간을 다른 procedures와 공유할 수 있음

다른 함수들이 한 메모리 공간을 나눠쓸 수 있다

단점 : 할당과 해제에 시간이 든다. runtime overhead

정적 변수에 비해서 접근 속도가 느림(간접 주소를 사용하기 때문)

Implicit Heap Dynamic Variables 묵시적 힙 동적 변수

: 값이 배정(assign)될 때 storage에 바인딩되는 변수

ex) 파이썬

`LIST = 10.2 5.1 0.0`

`LIST = 47`

장점 : 높은 유연성

단점 : runtime overhead. 컴파일 시간에 데이터타입 타입 검사 불가, 런타임에 해야 함.

Explicit Heap Dynamic Variables 명시적 힙 동적 변수

프로그래머에 의해 명시적으로 런타임에 지정된 변수 - 포인터를 사용한 동적할당 변수

데이터 타입은 컴파일 시간에 정해지고, storage에 바인딩/해제 - 실행시간

C에서 malloc, new, delete

장점 : 동적 구조를 만들 때 유용함. ex) linked list, tree 등 실행시간 도중에 크기가 변하는 것

단점 : 구현하기 어려움. 참조, 할당, 해제에 비용이 큼.

Java는 가비지 콜렉팅 지원

Type Checking 타입 검사

연산자와 피연산자가 호환가능한(compatible)한지 검사하는 것

Type Compatible 호환가능한 타입 : 연산자에 대해서 적합하거나, 컴파일러에 의해서 적합한 타입으로 바뀌는 것

coercion : 컴파일러에 의해 자동(강제) 타입 변환

Static type checking : 모든 변수가 정적 변수일 때 **컴파일 시간**에 타입검사

Dynamic type checking : **실행 시간**에 타입 검사. 파이썬은 동적 타입 검사를 함.

정적 타입 검사 : performance 면에서는 좋지만, 프로그램의 유연성이 떨어질 수 있음.

동적 타입 검사 : 가독성이 좋지 않음.

한 메모리공간에 다른 시간에 다른 타입을 허용하면 타입 검사가 복잡함. PASCAL의 Variant Record, C언어의 Union

Strong Type 강타입

강타입 언어의 2가지 조건

1. 모든 데이터 타입이 **정적으로 바인딩**되어야 함. - 타입이 항상 정적으로 유지.
2. **타입 에러가 컴파일 시간에 반드시 탐지**되어야 함.

C/C : 강타입 언어가 아님.

Ada : 강타입에 가까움

Pascal : 강타입에 가깝지만, variant record 때문에 아님

Java : 강타입에 가까움.

#Type Equivalence 타입 동등

type compatible보다 강한 조건.

Type compatible 호환 $\supset$ Type equivalence 동등

: 타입 강제변환(coercion) 없이 다른 값으로 assign 대입 가능할 때. type equivalence라고 한다.

1. Name type equivalence 이름 타입 동등

데이터 타입의 **이름**이 같아야지만 동등(대입 가능)

구현이 쉽지만, 굉장히 제한적

ex) Pascal

2. Structure Type equivalence 구조 타입 동등

구조가 같으면 동등(대입 가능)

구현하지 어렵지만, 유연함.

ex) Ada

Scope 스코프

Visible : 해당 문장에서 그 변수를 참조할 수 있는가?(볼 수 있는가?)

Scope : 변수가 보이는 visible한 범위(range)

지역변수 : 볼 수 있는데 해당 영역에서 선언이 되어있는 경우

비지역변수 : 볼 수 있는데 해당 영역에서 선언이 되어있지 않는 경우 ex)전역변수

함수가 중첩되어 있을 때 **비지역변수**에 접근하는 방법. Static vs Dynamic Scoping

1. Static Scoping 정적 스코프

함수가 중첩되어 있고, 비지역변수가 있을 때, 이를 참조하는 방법이 **코드의 구조**에 따라서 결정되는 방법
@ 컴파일 시간에

ALGOL60, Pascal, 파이썬이 사용/

```
procedure big ;  
  var x : integer ;  
  procedure sub1 ;  
    begin  
      ... x ....  
    end ;  
  procedure sub2;  
    var x : integer ;  
    begin  
      sub1 ;  
    end ;  
  begin  
    sub2 ;  
  end ;
```

< sub1안에 있는 x는 어떤 것을 참조하는가? 내 상위가 누구냐?

함수가 **호출된 위치가 아니라, 선언된 위치를 기준으로** 변수를 찾습니다.

procedure sub2 안에서는 또 다른 x가 선언되어 있습니다. 이 변수는 sub2 내에서만 유효한 새로운 변수입니다. sub2 안에서 sub1을 호출하더라도, sub1은 sub2의 x가 아닌, **정적 부모인 big에서 선언된 x를 참조**.

C언어에서 {}을 사용한 블록도 정적 스코프이다. ex)if문 안에서 변수 선언.

문제점 : 잘못된 함수 호출이 발생해도 컴파일러가 이를 감지하지 못할 수 있음.

과도한 데이터 접근; 비지역변수에 대한 접근은 최소화해야함. -> 변수에 대한 신뢰성에 영향
프로그램 구조를 안전하게 수정하기 어려움 ; 꼬일 수 있음

2. Dynamic Scope 동적 스코프

함수가 중첩되어 있고, 비지역변수가 있을 때, **함수의 호출 순서**에 따라서 상위가 결정되는 방법.

@ 런타임에 알 수 있음.

APL, SNOBOL4, 일부 LISP 방언에서 동적 스코핑 사용

장점 : 매개변수의 전달이 간편해짐. 순서에 따라 호출한 함수의 매개변수가 전부 전달됨.

문제점 : 호출 순서에 따라서 상위가 결정되기 때문에 함수 사이의 거리가 전혀 상관없게 됨. > 찾기가 힘들.

지역변수를 보호할 수 있는 방법이 없다.

가독성이 나쁨.

느리다.

★출제함★ Static scope일때의 출력을 쓰시오. Dynamic scope일때의 출력을 쓰시오

Scope and Lifetime

Scope : 볼 수 있는 **영역**

Lifetime : 변수가 메모리에 binding되는 **시간** - 함수가 호출되고 끝날때까지

Referencing Environment 참조 환경

: 문장에서 볼 수 있는 모든 이름(변수)의 모음

- @1 sub1의 x,y 와 example의 a,b
- @2 sub3의 x(지역이기 때문에), example의 a,b
- @3 sub2의 x, example의 a,b
- @4 example의 a, b

hidden variable : 구조상으로 볼 수 있지만
지역변수와 같은 이름이라 보이지 않는 변수

Named Constant 이름 상수

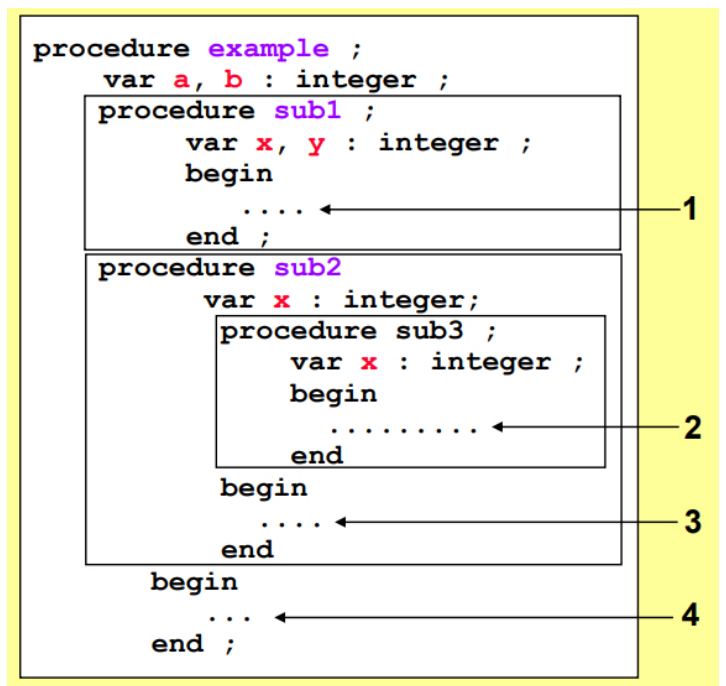
: 한번 바인딩되어서 메모리에 올라가는 상수

ex) const listlen=10 (Pascal)

Variable Initialization

: 변수가 기억공간(storage)에 바인딩되는 시점에 값이 같이 바인딩되는 것

Int a=10; //선언에 되면서 기억공간에 바인딩되는데, 값도 같이 바인딩되는 것



6장 - Data Types

#Data Type

데이터 값들의 집합. 미리 정의된 연산의 집합
얼마나 현실세계의 문제를 잘 표현할 수 있는지

FORTRAN 90이전 : 기본 구조만 지원. array로 linked list, non-linked list, binary tree 등 구현.

COBOL : 10진수 데이터값, 구조화된 데이터타입

PL/I : 다양한 데이터타입 지원

ALGOL-68 : 기본적 타입 지원, user-defined types 지원

Ada : 추상 데이터 타입 지원(직접 명시X)

#Primitive Data Type

Integer 정수형

가장 원시적인 타입

다양한 CPU는 integer 타입을 지원함. - byte, word, long word, quadword

Unsigned int 지원 - C/C++/C#

한 워드(word)안에 저장. ; 32비트 컴퓨터는 한 워드가 32비트

구현 : bit의 string으로 구현

2의 보수법을 통해서 연산

2의 보수 구하기 : 1의보수(비트반전) + 1

Floating point 부동소수점

대략적인 소수 표현. 정확한 소수 표현 불가 - 논리연산 시 주의해야!

IBM 704에서 처음으로

real(float)와 double을 지원함. single precision, double precision

IEEE 754에서 발표한 표준안

$f * 2^e$

	single pre	double pre
sign bit : 제일 앞 비트. 부호를 표시.	1비트	1비트
exponent(지수부) : 2^e 2의 e승	8비트	11비트
fraction(실수부) :	23비트	52비트

Decimal 10진수

BCD (Binary Coded Decimal) : 문자열과 비슷하게 저장
정교한 10진수 값을 표현할 수 있음.

packed : 1바이트에 숫자 2개 표현. 압축해서 표현
unpacked : 1바이트에 숫자 1개 표현

Boolean 불리언

가장 간단한 타입

2개의 원소 : True or False - 가독성을 위한 조치

비트 1개만 쓰지만, Access하기 어렵기에 Byte단위로 쓴다.(8비트를 씀)

Character 문자

숫자로 저장된 문자타입

C/C++ : char

ASCII코드: 1바이트 0~127까지 값으로 저장

유니코드 : 2바이트 - Java JS 파이썬 등

Character String 문자열

character의 연속적인 집합

디자인 고려 사항 : 문자열이 primitive type인가 문자의 array형인가?

문자열이 동적/정적 길이를 가지는가?

1. char의 array로의 문자열

char의 array로 저장됨. Pascal, Modula-2, C/C++, Ada

C언어에서의 <string.h> - NULL 문자(0)를 사용해서 문자열의 끝을 나타냄

2. primitive 문자열

작성력(Writability)가 향상됨.

할당, 문자열 연산, 연결, 하위문자열 참조(슬라이싱)

Java, 파이썬...

1. Static Length String 정적 길이 문자열

선언과 동시에 길이가 정해짐.

FORTAN77, FORTRAN90, COBOL, Ada...

2. Limited Dynamic Length String 고정된 동적 길이 문자열

C언어에서 char str[20]으로 선언하면 최대 20인 문자열이지만 그 안에서는 가변적으로 길이 변함.

3. Dynamic Length String 동적 길이 문자열

최대 길이 제한없음. 유연함, 동적이므로 저장공간 할당/회수에 따른 overhead

>> Ada95 이후는 3가지 모두 지원!

문자열은 작성력에 큰 영향을 미침 - C언어에서는 strcpy()를 안쓰면

primitive로 지원하는 것이 비용도 크지 않을뿐더러 Writability에 좋다!

- 패턴 매칭, 접합 현대언어에서는 대부분 지원

구현 :

Descriptor : 변수의 특성 집합 ; 메타데이터!

Static에 필요한 descriptor : 문자열 + 주소 + 길이

Dynamic에서 필요한 descriptor: 문자열 + 주소 + 최대길이 + 현재길이

가능한 2가지 방법 :

1) **linked list** : 느리다. 더 많은 stroge 차지

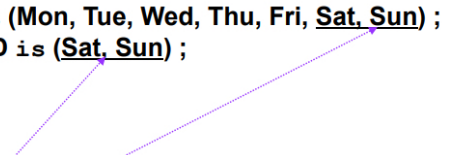
2) **전체를 copy**해서 새로운 공간에 할당. reference가 빠름, 적은 공간, 할당/해제에 비용이 듦

User-Defined Ordinal types 사용자 지정 순서 타입.

: 가능한 값이 양의 정수와 연관된 타입

Enumeration, Subrange types 두 가지가 있다.

```
type DAYS is (Mon, Tue, Wed, Thu, Fri, Sat, Sun) ;  
type WEKEND is (Sat, Sun) ;  
int i ;  
DAYS a ;  
a = Mon ;
```



Enumeration Types

: 모든 가능한 값(symbolic constants 상징적인 상수)들이 열거되는 타입.

Mon, Tue,... 이 각각 0, 1, 2... 와 연결됨

디자인 고려사항 : overload를 할 것인가? 다른 타입에서 중복을 허용할 것인가? ex) Sat, Sun
타입 검사를 어떻게 할 것인가?

Pascal : 하나의 enum에서 쓰였으면 다른 타입에서는 재사용할 수 없음.

인덱스나, 반복문의 변수, 조건문 case에서 선택자로 쓰일 수 있음.

비교 연산도 가능함.

C : 묵시적으로 정수형으로 바뀜

enum type을 사용하는 이유?

Readability 향상. 직관적임

숫자형과 비교하면 산술 연산 불가, 오류 찾기 쉬워짐.

Subrange Types 부분 범위 타입

: 순차적 타입의 연속적인 부분 타입

가독성과 안정성이 높음

but, Ada95 이상으로는 왜인지 지원을 안 함.

User-defined Ordinal types의 구현

Enum type - 0이상 정수로 구현

Subrange - 범위 검사를 제외하면 정수형과 비슷하게 구현

#Array Types 배열 타입

동질적인 데이터의 묶음(Homogeneous aggregate)

첫 번째 원소와 상대적인 위치로 구분

일반적으로 subscripts 첨자 []로 구분.

배열의 원소의 위치를 구분하기 위해서는 실행시간에 계산이 따로 필요함

디자인 고려사항 :

* 첨자(subscripts)에는 어떤 타입이 유효한가? ex) int, ordinal types...

* 첨자는 언제 바인딩 되는가? ex) 실행시간 vs 컴파일시간

* 배열에 대한 할당은 언제 발생하는가? ex) 실행시간 vs 컴파일시간

* 몇 개까지 첨자를 허용할 것인가? ex) 차원

* 공간이 할당될 때 초기화를 하는가?

* 슬라이싱이 가능한가?

Array 참조

2개로 이루어짐

aggregate name 배열 이름 + index(subscripts) 인덱스(첨자) : a[100]

배열이름[인덱스 값] -> 원소 : 이렇게 맵핑이 된다!

소괄호 vs 대괄호

FORTRAN, PL/I

소괄호로

Pascal, C...

대괄호로

첨자의 바인딩

: 주로 정적으로 이루어짐

첨자 범위의 하한

C언어 : 0부터

FORTRAN 1,2,4 : 1부터

FORTRAN 77,90 : 1부터이지만 수정 가능

첨자 범위 바인딩과 저장공간 바인딩에 따른 4가지 구분

1. Static Array

: 첨자의 범위와 기억공간(storage) 할당이 모두 정적. 실행시간 전에 binding이 모두 끝남. 효율적.

FORTRAN 77

2. Semistatic Array (Fixed stack-dynamic array)

고정 스택동적 배열

: 첨자의 범위는 정적이지만, 기억공간에 대한 할당이 동적(@ 실행시간)

Pascal의 local array

3. Semidynamic Array (Stack-dynamic array)

: 첨자의 범위, 기억공간에 대한 할당이 모두 동적(@실행시간). 한번 바인딩 되면 배열의 수명동안 고정

C언어, FORTRAN90 - c언어의 malloc

4. Dynamic Array (Heap dynamic array)

: 첨자의 범위, 기억공간에 대한 할당이 모두 동적. 실행 시간동안 변경도 가능

ALGOL 68

유연하지만 느림.

Heterogeneous Array 이중 배열

: 배열의 원소가 같은 타입일 필요가 없음

JS, 파이썬 등 스크립트 언어에서 주로 지원

> Heap Dynamic array

array의 첨자수(차원)

FORTRAN1 : 3 FROTRAN 77,90 : 7 다른언어 : 제한 없음

C언어는 1개의 첨자(1차원)만 지원한다. but, array의 요소로 array를 둘 수 있다.

int mat[5][4]; << mat[5]가 이름이고 [4]가 첨자이다!

array의 초기화

FORTRAN 77 : 모든 데이터가 정적으로 할당됨. 로드시간 초기화 허용됨.

C : 정적배열은 초기화 가능, 동적배열은 불가능. int list[]={4,5,7,83} < 가능! malloc쓴 배열에는 불가능

Pascal, Modular-2 : 배열 초기화 허용 X

Ada : 두 가지 방법으로 초기화 가능. 순서대로 넣기 and 요소 번호로 직접 넣기

array의 연산

FORTRAN : elemental이라는 연산 가능. 합, 할당, 산술, 관계, 논리연산이 모든 크기에 대해 가능
APL : 사칙연산, 벡터, 행렬, 스칼라 연산 지원, 내적, 외적 지원
C : array 연산 지원 안함
Java/C++ : 메서드를 통해 일부 지원
Python : 배열 접합, 비교, 추가 일부 지원

슬라이스

array의 일부분만 자르기
FORTRAN : 3x3이 있을 때 MAT(1:3,2)
파이썬 : vector[0:7:2] 시작;끝;점프

array의 구현

1차원 array :

$$\begin{aligned} \text{address}(\text{list}[k]) &= \text{address}(\text{list}[1]) + (k-1) * \text{element_size} \\ &= (\text{address}(\text{list}[1]) - \text{element_size}) + (k * \text{element_size}) \end{aligned}$$

constant, so that can be computed at compile time

Compile-time descriptor : 배열 + 요소 타입 + 인덱스타입 + 최저 인덱스 + 최고 인덱스

다차원 배열의 구현

Row major order : 행 순서로 대부분의 언어
Column major order : 열 순서로 - FORTRAN

3	4	7
6	2	5
1	3	8

Access Function 접근 함수

다차원 배열에서 요소에 접근하기 위한 함수

- row major order : 3,4,7, 6,2,5, 1,3,8
- column major order : 3,6,1, 4,2,3, 7,5,8

The access function for two-dimensional arrays stored in row major order
$$\begin{aligned} \text{location}(\text{a}[\text{i},\text{j}]) &= (\text{address of } \text{a}[1,1] + (((\text{i}-1)*\text{n}) + (\text{j}-1)) * \text{element_size}) \\ &= (\text{address of } \text{a}[1,1]) - ((\text{n}+1) * \text{element_size}) + ((\text{i} * \text{n} + \text{j}) * \text{element_size}) \end{aligned}$$

constant

Associative Array 연상 배열

Keys-value 가 쌍으로 이루어져 있는 배열

Python : 딕셔너리

Java, C++ : Hash map

Perl : Hash

장점 : 원소에 접근할 때 직관적이고 효율적이다.

-> 일부 원소에만 접근할 때 유리. 전체 원소에 접근할 때는 일반 배열이 좋음.

Record Types 레코드 타입

다양한(Heterogeneous) 타입의 집합. 각 원소는 멤버 이름으로 구분됨

COBOL에서 처음 등장함.

레코드의 구조

COBOL에서는 맨 앞의 01, 02, 05같은 숫자로 level을 구분함.

ADA에서는 레코드 내부에 레코드를 선언함으로써 계층 만들. 모든 레코드는 이름이 있어야 함.

FORTRAN/C에서는 레코드들을 각각 선언하고, 레코드 안에 중첩(nest)시킴.

★C언어에서의 구조체! Struct이다.

레코드의 문법

COBOL : OF를 사용함. ex) MIDDLE OF EMPLOYEE-NAME OF EMPLOYEE-RECORD

다른 언어 : 점(.)으로 구분함. dot notation.

참조 방법

Fully qualified reference 완전 자격 참조 : 참조할 때 항상 fullname을 씀 C언어

Elliptical reference 생략 참조 : 생략할 수 있음 --- but, 헷갈릴 수 있음. Readability 가독성 저하.

레코드 간의 연산

Pascal, Modular-2, C : 할당(대입) 가능

Ada : 할당, 비교 가능

COBOL : MOVE CORRESPONDING을 사용하면 공통적인 부분만 복사 가능!

레코드의 구현

인접한 메모리에 저장

offset 주소에서 시작해 각 필드의 주소를 찾기.

word alignment : 접근을 효율적으로 하기 위해서 **word 단위로 접근**함. 빈 공간은 padding

char c;가 1000번지에 있고 그 다음에 int i;가 오면 char는 1바이트만 쓰지만 1004바이트에 할당

UNION Types 공용체

: 다른 시간에 다른 값을 저장할수 있게 하는 데이터타입 @실행시간

: 동일한 데이터공간에 저장되지만, 시간에 따라 어떤 자료형을 쓸지 달라짐. 스토리지 할당은 제일 큰 것 기준.

디자인 :

타입검사를 해야 하는가? (무조건 동적)

FORTAN : EQUIVALENCE라는 타입. 타입검사를 하지 않음.

ALGOL 68 : tag를 활용해 현재 어떤 데이터 타입을 가지고 있는지 표기함(**Discriminated Union**)

Conformity clause(적합성 문장) : 실행시간에 변수가 어떤 데이터 타입인지를 검사해 각 경우에 따라서 다르게 처리하기

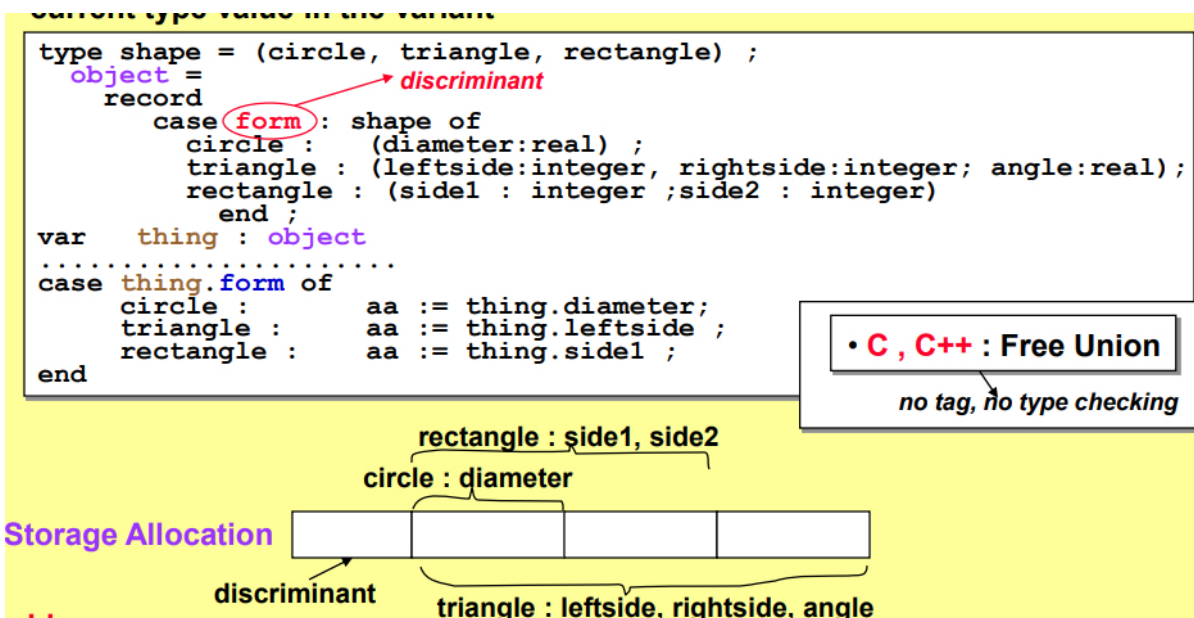
Alias vs Union :

Alias는 한 공간을 서로다른 변수에서 접근하는 것

Union은 한 공간을 다른 타입이 공유하는 것. 동시에 존재 X

Pascal : Variant Record라는 이름. 레코드의 한 타입. Discriminant Union 방식 채택

form이 discriminant. 값에 따라 circle/triangle/rectangle 중 1개만 사용. 공간 할당은 제일 큰 tri 기준.



문제 : 신뢰성 저하. 타입검사 어려움 특히, C/C++ 타입검사 안함. 안정성 낮아짐.
>> 요즘에는 많이 쓰지 않음

구현 : 모든 변수를 **같은 주소**에 할당. 할당할 때는 가장 큰 멤버를 기준으로 할당.

SET Types 집합

: 순서가 없는 집합. 구별되는 값들만으로 이루어져 있음(중복이 없음. distinct)

디자인 :

최대 원소의 base type을 몇 개로 할 것인가? ???

Pascal :

대개 100개 이하의 base set을 갖도록 설계가 되어있음

구현 :

집합은 실제로는 bit string으로 저장됨. 계산의 편의를 위해서
집합 연산이 array 연산보다 효율적이다. >> bit 연산이기 때문!

set union(합집합) : 논리 OR연산

member check(in 연산 들어있는가?) : 논리 AND 연산

Pointer and Reference Types 포인터와 참조 타입

: 메모리 주소와 nil 값(현재 참조할 수 없다는 것을 나타내는 값)으로 구성되는 자료형

간접 주소, 동적 할당을 위해서 쓰임

PL/I이 처음 포인터 사용

디자인 :

포인터 변수의 scope와 lifetime이 어떻게 되는가?

포인터가 참조하는 동적 변수의 lifetime은?

★이 포인터가 어떤 데이터타입을 가리키지?? >> 타입 검사와 연관 애가 뭘 가리키지? 알기 어려움

포인터 연산자

1. object의 주소를 포인터 변수에 넣겠다

```
int * aa, bb, cc;      //aa를 포인터 변수로 선언하겠다!  
aa = &bb;            //bb의 주소를 포인터 변수인 aa에 넣겠다.
```

2. 역참조(dereferencing)

```
cc = *aa;            //aa포인터 변수를 역참조해서 그 값을 cc에 넣어라.
```

Reference type : 실제 데이터를 저장하지 않는 자료형

#PL/I에서의 포인터 문제

타입 검사

domain type :(포인터가 가리킬 수 있는 타입)

PL/I에서는 가리킬 수 있는 포인터의 제한이 없음. >> C언어는 int형 포인터는 int만 가리킬 수 있음!

Dangling Pointer 댕글링 포인터

이미 해제(회수)된 메모리를 가리키는 것.

Lost Objects (Garbage) 로스트 오브젝트

어떤 변수가 할당을 하고, 해제하지 않은 채로 새로운 공간을 할당하면, 그 공간에 접근을 못 함.

#Pascal에서의 포인터

동적할당 할 때만 사용됨!

할당 : *new*

해제 : *dispose*

그런데.... **dispose** 함수에 문제가 있었다! - 항상 dangling pointer 문제가 생김

- * *dispose*를 무시해버림

- * 언어 자체에서 *dispose*를 빼버리자

- * dangling pointer를 허용해버리자

- * 동적 변수가 파괴될 때 *nil*을 가리키게 하자 - overhead... 해제될때마다 자원 사용

- * 스택을 사용해서...?

#C에서의 포인터

어셈블리어의 주소처럼 사용할 수 있음

dangling pointer와 lost object에 대한 해결책이 아예 없음!!!

포인터에 대한 산술연산 허용

- & 변수의 주소값을 참조

- * 역참조

- +, - 다음 메모리를 가리켜라

ptr+idx : 메모리 단위로 움직임

goto문과 pointer는 실패한 설계이다... > 현대는 포인터 잘 안씀

#Reference Type 레퍼런스 타입

: 주소가 아닌, object나 값을 가리킴

C++에서 도입.

실제 구현은 포인터로 되어 함수 안팎에서 외부의 값을 변경시키기 쉬워짐. 양방향 통신이 가능해짐!!

원래 변수와 레퍼런스 타입으로 가리키고 있는 변수는 **alias** 관계이다

#포인터에 대한 구현

워드 사이즈만큼의 크기로 변수를 만든다. 그렇게 어려운 구현은 아님!

#Dangling Pointer에 대한 해결책

1. Tombstone Approach 톰스톤 접근방법

tombstone이라는 특수한 cell을 만들어 이중 포인터처럼 구현을 하는 것.

만약에 만약에 할당이 해제되더라도 tombstone을 가리키게 됨. tombstone은 *nil*을 가리킴.

>>시간과 공간이 모두 듬.

C++과 구형 MAC 컴퓨터에서 사용했음.

2. Locks-and-key Approach 자물쇠-키 접근방법

포인터와 동적할당된 메모리 공간에 작은 공간을 만들어 같은 값을 넣어놓는다.

key-포인터 lock-동적변수의 메모리

참조할 때 두 값을 비교한다.

동적할당된 변수가 해제되면 lock 값이 사라진다.

#Heap 관리와 Lost Object 해결법

모든 힙 공간이 고정된 크기로 할당을 한다면... LISP 언어에서 사용

1. Reference Counter Approach 참조 카운터 접근법

모든 힙 공간의 메모리 셀에 조그만 공간을 만들어 몇 개의 포인터가 변수를 가리키고 있는지 센다.

0이 되면 해제한다. >> **incremental reclamation**(순차적 회수 : 그때그때 회수)

문제점 : 1. 카운터를 저장할 공간이 필요하다 2. 실행시간에 카운터를 동작시켜야 한다. 3. 순환 참조시 영원히...

2. Garbage Collection of fixed size cell 고정된 셀 사이즈에서의 가비지 콜렉션

batch relamation(일괄 회수)

모든 가용 셀이 할당될 때까지 reclamation(재활용)하지 않음.

힙의 각 셀은 indicator bit 한 비트를 가지고 있어 쓰레기이면 1, 아니면 0 인 방식으로 검사를 함.

작동 방식

1. 모든 셀이 garbage indicator를 가지고 있음.
2. 모든 포인터를 검사해서 메모리 셀을 가리키고 있으면 그 셀은 쓰레기가 아닌 것으로 표시함
3. 쓰레기를 수집하고, 아닌 것들은 다시 가능 공간으로 돌려줌

문제점 : 필요할 때 최악으로 작동할 수 있음. mark bit에 공간이 림고, 추적하는데 시간이 듬.

3. Garbage Collection of variable size cell 다양한 셀 사이즈에서의 가비지 콜렉션

기존 가비지 콜렉션의 문제에 더해서...

1. 셀이 어디서 시작하고 끝나는지 알기 어려움! -> 시작과 끝을 indicator로 표시.
2. 힙공간에 할당은 되어있지만, 포인터로 가리켜지지 않은 메모리가 있을수 있음. -> 내부 포인터 (가짜 포인터)를 가리키게 함
3. Fragmentation(파편화)문제가 생길 수 있음 -> 셀 위치 바꾸기등등... 방법이 있지만 overhead